



# DATA MINING & DATA WAREHOUSING

|              | ASSIGNMENT DETAILS                                                    |
|--------------|-----------------------------------------------------------------------|
| NAME         | ARKAPRAVA ROY                                                         |
| SUBJECT      | DATA MINING & DATA WAREHOUSING                                        |
| STREAM       | B-TECH ( IT )                                                         |
| SESSION      | 2023-2027                                                             |
| COLLEGE NAME | GOVERNMENT COLLEGE OF ENGINEERING AND TEXTILE TECHNOLOGY<br>SERAMPORE |

# What is a Data Stream?

A **Data Stream** is a continuous sequence of data items that arrive over time at a rapid rate.

Unlike traditional databases, data streams are **continuous, unbounded (potentially infinite), and generated in real time**.

## Examples

- Stock market prices
- Sensor readings
- Network traffic
- Social media posts
- Online transaction logs

Example:

Temperature Sensor:

25°C → 26°C → 25°C → 27°C → 28°C → ...

The data keeps arriving continuously, forming a **data stream**.

## Characteristics of Data Streams

- ✓ Continuous flow of data
- ✓ Generated in real time
- ✓ Very large or infinite size
- ✓ Fast arrival rate
- ✓ Usually processed only once

## Why Traditional Databases Don't Work Well

Traditional databases assume:

```
Store Data
  ↓
Process Later
```

But for data streams:

```
Data Arrives Continuously
  ↓
Process Immediately
```

because storing everything may be impossible.

## Applications

- Weather monitoring
- IoT devices
- Fraud detection
- Traffic monitoring
- Financial market analysis

# Challenges of Stream Data Mining

Data stream mining is difficult because data arrives continuously and rapidly. The main challenges are:

## 1. Infinite Volume of Data

Data streams may never end, so it is impossible to store the entire stream.

Sensor Data:  
1, 2, 3, 4, 5, ...

The data keeps coming indefinitely.

## 2. Limited Memory

Since the stream is huge, only a small portion of data can be kept in memory.

## 3. High Speed of Data Arrival

Data arrives very quickly and must be processed in real time.

Examples:

- Network traffic
- Stock market transactions

## 4. Single-Pass Processing

Usually, each data item can be examined only once because storing and rescanning the stream is impractical.

Read → Process → Discard

## 5. Real-Time Response Requirement

Results must be produced immediately for applications such as:

- Fraud detection
- Traffic monitoring
- Online recommendations

## 6. Concept Drift

The characteristics of the data may change over time.

Example:

Customer buying patterns in 2025  
≠  
Customer buying patterns in 2026

Mining algorithms must adapt to these changes.

## 7. Data Quality Issues

Streams may contain:

- Missing values
- Noise
- Duplicate records
- Inconsistent data

## 8. Resource Constraints

Processing must be done with limited:

- CPU time
- Memory
- Storage



# What is Synopsis in Stream Data Mining?

## Definition

A **Synopsis** is a compact summary or compressed representation of a data stream that allows efficient analysis without storing the entire stream.

Since stream data is continuous and potentially infinite, storing all data is impractical. Instead, a synopsis keeps only essential information.

## Example

Suppose a stream contains:

10, 12, 11, 15, 14, 13, 16, ...

Instead of storing every value, a synopsis may store:

```
Count = 8  
Average = 13  
Maximum = 16  
Minimum = 10
```

This summary is called a **synopsis**.

## Why is Synopsis Needed?

Stream data mining faces:

- Infinite data volume
- Limited memory
- High data arrival rate

A synopsis helps by:

- ✓ Reducing storage requirements
- ✓ Enabling fast processing
- ✓ Supporting approximate query answering

## Synopsis Data Structures

Synopsis data structures are specialized structures used to maintain compact summaries of data streams.

Common synopsis structures include:

## 1. Sampling

Stores only a subset of stream elements.

Example:

Stream:  
1, 2, 3, 4, 5, 6, 7, 8, 9

Sample:  
2, 5, 8

## 2. Histograms

Store frequency distributions of values.

Example:

| Range | Count |
|-------|-------|
| 0–10  | 15    |
| 11–20 | 25    |
| 21–30 | 10    |

## 3. Wavelets

Compress data while preserving important trends and patterns.

Used for efficient approximation.

## 4. Sketches

Probabilistic summaries that estimate frequencies and counts.

Used when exact answers are not required.

## 5. Reservoir Sampling

Maintains a fixed-size random sample from an infinite stream.

# Reservoir Sampling

## Definition

Reservoir Sampling is a stream data processing technique used to maintain a **random sample of fixed size (k)** from a data stream of unknown or infinite length.

Since the stream may be very large, we cannot store all elements. Reservoir sampling keeps only a small representative sample.

## Working

Suppose:

Stream:  
1, 2, 3, 4, 5, 6, 7, 8, ...

Sample size:

$k = 3$

Steps:

1. Store first 3 elements in the reservoir.

Reservoir = {1, 2, 3}

2. For each new element, randomly decide whether to replace an existing element in the reservoir.
3. Continue until the stream ends.

## Advantages

- ✓ Fixed memory usage
- ✓ Random and unbiased sample
- ✓ Suitable for infinite streams

# Sliding Window Model

## Definition

The Sliding Window Model is a stream processing technique that considers only the most recent data items within a fixed-size window and discards older data.

It assumes that recent data is more important than old data.

## Working

Suppose window size = 4

Stream:

1 2 3 4 5 6 7

Initial window:

1 2 3 4

When 5 arrives:

2 3 4 5

When 6 arrives:

3 4 5 6

The window continuously "slides" forward.

## Advantages

- ✓ Focuses on recent data
- ✓ Reduces memory requirements
- ✓ Adapts to changing patterns (concept drift)

# Application Fields for Similarity Search in Time-Series Analysis

## Definition

Similarity search in time-series analysis is used to find time-series data that has patterns similar to a given query sequence.

For example:

Query:  
10, 15, 20, 18, 25

Find other sequences  
having a similar trend.

## Application Fields

### 1. Stock Market Analysis

Used to find historical stock price patterns similar to current market behavior.

Example:

Current stock trend  
↓  
Find similar past trends  
↓  
Predict future movement

### 2. Weather Forecasting

Compare current weather patterns with historical weather data.

Example:

Temperature pattern  
↓  
Find similar past patterns  
↓  
Forecast weather

### 3. Medical and Healthcare

Used in analyzing:

- ECG signals
- EEG signals
- Heart rate monitoring

Example:

Patient ECG  
↓  
Compare with known disease patterns

## 4. Speech Recognition

Voice signals are time-series data.

Used to identify words and speakers by comparing signal patterns.

## 5. Sensor and IoT Monitoring

Used in:

- Industrial machines
- Smart homes
- Environmental monitoring

Example:

Machine vibration pattern  
↓  
Compare with fault patterns  
↓  
Detect equipment failure

## 6. Network Traffic Analysis

Detect unusual network behavior by comparing traffic patterns.

Applications:

- Intrusion detection
- Cybersecurity
- Network monitoring

## 7. Financial Fraud Detection

Compare transaction patterns with known fraudulent patterns.

Example:

Credit card transactions  
↓  
Similarity search  
↓  
Fraud detection

# Why Normalization is Necessary for Similarity Search?

## Definition

Normalization is the process of scaling data values to a common range before performing similarity search.

In time-series similarity search, two sequences may have the **same shape or trend** but different magnitudes. Without normalization, they may appear dissimilar.

## Example Without Normalization

Consider two time-series:

A = [10, 20, 30, 40]

B = [100, 200, 300, 400]

Although B is just a scaled-up version of A, a direct comparison will show a large difference.

## Example With Normalization

After normalization:

A → [0.25, 0.50, 0.75, 1.00]

B → [0.25, 0.50, 0.75, 1.00]

Now both sequences have the same pattern and can be correctly identified as similar.

## Why It Is Needed

### 1. Removes Scale Differences

Two series may have different units or magnitudes.

Example:

Temperature: 20, 25, 30

Temperature: 200, 250, 300

Trend is identical, scale is different.

### 2. Focuses on Shape Instead of Magnitude

Similarity search usually aims to compare:

Pattern  
Trend  
Behavior

rather than absolute values.

### 3. Improves Accuracy

Without normalization, distance measures (Euclidean distance, etc.) may be dominated by large values.

### 4. Handles Different Measurement Units

Example:

Temperature in °C

Temperature in °F

Normalization allows meaningful comparison.

## Min-Max Scaling and Z-Score Normalization

These are two common **data normalization techniques** used in data mining and time-series analysis.

### 1. Min-Max Scaling

#### Definition

Min-Max Scaling transforms data values into a specified range, usually [0,1].

Formula:

$$v' = \frac{v - \min(A)}{\max(A) - \min(A)}$$

Where:

- $v$  = original value
- $v'$  = normalized value
- $\min(A)$  = minimum value of attribute
- $\max(A)$  = maximum value of attribute



## Example

Data:

10, 20, 30, 40

For value = 20

$$v' = \frac{20 - 10}{40 - 10} = \frac{10}{30} = 0.33$$

## Characteristics

- ✓ Values lie between 0 and 1
- ✓ Preserves relationships among data values
- ✗ Sensitive to outliers

# Z-Score Normalization

## Definition

Z-Score Normalization transforms data based on its mean and standard deviation.

It converts values into the number of standard deviations from the mean.

$$z = \frac{x - \mu}{\sigma}$$

Formula:

$$z = \frac{v - \mu}{\sigma}$$

Where:

- $v$  = original value
- $\mu$  = mean
- $\sigma$  = standard deviation
- $z$  = normalized value

## Example

Suppose:

Mean = 50  
Standard Deviation = 10  
Value = 70

$$z = \frac{70 - 50}{10} = 2$$

Meaning:

70 is 2 standard deviations above the mean.

## Characteristics

- ✓ Mean becomes 0
- ✓ Standard deviation becomes 1
- ✓ Less affected by extreme values

# Class Imbalance Problem

## Definition

The **Class Imbalance Problem** occurs when the number of instances in one class is much larger than the number of instances in another class.

In such datasets, machine learning models tend to favor the majority class and perform poorly on the minority class.

## Example

Suppose a fraud detection dataset contains:

| Class                | Records |
|----------------------|---------|
| Genuine Transactions | 9900    |
| Fraud Transactions   | 100     |
| 99% Genuine          |         |
| 1% Fraud             |         |

This is a **class-imbalanced dataset**.

## Why is it a Problem?

A classifier may simply predict:

Always Predict Genuine

Accuracy:

$9900 / 10000 = 99\%$

Looks excellent, but:

Fraud Detection = 0%

The model completely fails to identify fraud cases.

## Effects of Class Imbalance

### 1. Biased Learning

Model becomes biased toward the majority class.

### 2. Poor Minority-Class Detection

Rare but important classes are often misclassified.

### 3. Misleading Accuracy

High accuracy may not indicate good performance.

# Outlier Analysis in Data Mining

## Definition

**Outlier Analysis** is the process of identifying data objects that significantly differ from the majority of the data.

These unusual data points are called **outliers**.

## Example

Consider the ages:

20, 22, 21, 23, 24, 25, 90

Here:

90

is an outlier because it is far away from the other values.

## Why Detect Outliers?

Outliers may indicate:

- Fraudulent transactions
- Network intrusions
- Medical abnormalities
- Data entry errors
- Rare but important events

## Methods for Outlier Detection

### 1. Statistical Method

Assumes data follows a statistical distribution.

Objects far from the mean are considered outliers.

Example:

Mean = 50

Most values near 50

Value = 150

Outlier detected using Z-score or standard deviation.

### Advantage

- Simple and fast.

### Disadvantage

- Requires distribution assumptions.

## 2. Distance-Based Method

Measures distance between data points.

A point is an outlier if it has very few neighboring points within a specified distance.

Example:



The isolated point is an outlier.

### Advantage

- Easy to understand.

### Disadvantage

- Expensive for large datasets.

## 3. Density-Based Method

Compares local density of a point with densities of its neighbors.

If density is much lower, the point is an outlier.

Example:

Cluster 1: Dense



The isolated point has low density and is considered an outlier.

### Example Algorithm

- LOF (Local Outlier Factor)

## 4. Clustering-Based Method

Clusters are formed first.

Points that do not belong to any cluster or belong to very small clusters are treated as outliers.

Example:

```
Cluster A: 100 points  
Cluster B: 80 points  
Single point: 1
```

The single point is an outlier.

### Example Algorithms

- K-Means
- DBSCAN

## 5. Classification-Based Method

Uses machine learning models to classify data as:

```
Normal  
or  
Outlier
```

Often used when labeled data is available.

### Examples

- Decision Trees
- SVM
- Neural Networks

# Association Rule Mining

**Association Rule Mining** is a data mining technique used to find interesting relationships, patterns, or associations among items in a transaction database.

It answers questions like:

"If a customer buys X,  
what else is he likely to buy?"

## Real-Life Example

Suppose a supermarket records the following transactions:

| Transaction ID | Items Bought        |
|----------------|---------------------|
| T1             | Bread, Butter       |
| T2             | Bread, Milk         |
| T3             | Bread, Butter, Milk |
| T4             | Butter, Milk        |
| T5             | Bread, Butter       |

From these transactions, we may discover:

$\text{Bread} \rightarrow \text{Butter}$

Meaning:

Customers who buy Bread often buy Butter.

This is called an **Association Rule**.

## Components of Association Rules

An association rule is written as:

$X \rightarrow Y$

Where:

- $X$  = Antecedent (Left-hand side)
- $Y$  = Consequent (Right-hand side)

Example:

$\text{Bread} \rightarrow \text{Butter}$

- Bread = Antecedent

- Butter = Consequent

Meaning:

If Bread is purchased, Butter is likely to be purchased.

## Important Measures

### 1. Support

Support tells how frequently an itemset occurs in the database.

#### Formula

$$\text{Support}(X \rightarrow Y) = \frac{\text{Count}(X \cup Y)}{\text{Total Transactions}}$$

#### Example

Bread and Butter appear together in:

T1, T3, T5

Total transactions = 5

Support:

$$= \frac{3}{5} = 0.6 = 60\%$$

#### Interpretation

60% of all transactions contain Bread and Butter together.

### 2. Confidence

Confidence tells how often Y occurs when X occurs.

#### Formula

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Count}(X \cup Y)}{\text{Count}(X)}$$

#### Example



Bread appears in:

T1, T2, T3, T5

= 4 transactions

Bread and Butter together:

T1, T3, T5

= 3 transactions

Confidence:

$$= \frac{3}{4} = 75\%$$

### Interpretation

Among customers buying Bread,  
75% also buy Butter.

## 3. Lift

Lift measures how strong the rule is.

### Formula

$$Lift(X \rightarrow Y) = \frac{Confidence(X \rightarrow Y)}{Support(Y)}$$

Interpretation:

- Lift > 1 → Positive association
- Lift = 1 → Independent
- Lift < 1 → Negative association

## Steps of Association Rule Mining

### Step 1

Find Frequent Itemsets

Example:

Bread  
Butter

Milk  
Bread, Butter  
Bread, Milk  
Butter, Milk

## Step 2

Remove Itemsets Below Minimum Support

Suppose:

Minimum Support = 40%

Only keep itemsets meeting this threshold.

## Step 3

Generate Association Rules

Example:

Bread → Butter  
Butter → Bread  
Milk → Bread

## Step 4

Calculate Confidence

Keep only rules satisfying:

Minimum Confidence

# Apriori Algorithm

The **Apriori Algorithm** is the most famous algorithm used for **Association Rule Mining**. It finds **frequent itemsets** in a transaction database and then generates association rules.

Apriori is an algorithm that discovers frequent itemsets and generates association rules using the **Apriori Property**.

## Apriori Property

**If an itemset is frequent, then all of its non-empty subsets must also be frequent.**

Example:

If

`{Bread, Butter, Milk}`

is frequent, then

`{Bread, Butter}`  
`{Bread, Milk}`  
`{Butter, Milk}`  
`{Bread}`  
`{Butter}`  
`{Milk}`

must also be frequent.

## Key Terms

### 1. Itemset

A collection of one or more items.

Example:

`{Bread}`  
`{Bread, Butter}`  
`{Bread, Butter, Milk}`

### 2. Support

Frequency of an itemset in the database.

$$Support(X) = \frac{\text{Number of transactions containing X}}{\text{Total transactions}}$$

### 3. Confidence

Measures how often Y occurs when X occurs.

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

## Example Database

#### TID Items

T1 A, B, C

T2 A, B

T3 A, C

T4 B, C

T5 A, B, C

Let:

Minimum Support Count = 2

### Step 1: Generate Frequent 1-Itemsets (L1)

Count each item.

#### Item Count

A 4

B 4

C 4

All counts  $\geq 2$

Therefore:

$L1 = \{A\}, \{B\}, \{C\}$

### Step 2: Generate Candidate 2-Itemsets (C2)

Join L1 with itself:

AB

AC

BC

Count supports.

**Itemset Count**

|    |   |
|----|---|
| AB | 3 |
| AC | 3 |
| BC | 3 |

All satisfy minimum support.

Therefore:

$L_2 = \{AB, AC, BC\}$

## Step 3: Generate Candidate 3-Itemsets (C3)

Join frequent 2-itemsets.

Possible candidate:

ABC

## Pruning Using Apriori Property

Check all subsets of ABC:

|    |   |
|----|---|
| AB | ✓ |
| AC | ✓ |
| BC | ✓ |

Since all subsets are frequent:

ABC is kept

Count support:

**Itemset Count**

|     |   |
|-----|---|
| ABC | 2 |
|-----|---|

Since count  $\geq 2$ :

$L_3 = \{ABC\}$

## Step 4: Generate Candidate 4-Itemsets

Try joining L3 with itself.

No larger itemset can be formed.

Algorithm stops.

## Final Frequent Itemsets

### L1

A  
B  
C

### L2

AB  
AC  
BC

### L3

ABC

## Candidate Generation and Pruning

This is the heart of Apriori.

### Candidate Generation

Join frequent itemsets:

AB + AC  
          ↓  
ABC

### Pruning

Remove candidate if any subset is infrequent.

Example:

Suppose

AB = Frequent  
AC = Frequent  
BC = Not Frequent

Then

ABC

cannot be frequent.

So remove it immediately.

This reduces computation.

## Association Rule Generation

From frequent itemset:

ABC

Generate:

AB → C  
AC → B  
BC → A  
A → BC  
B → AC  
C → AB

## Confidence Calculation

For rule:

AB → C

$$\begin{aligned} \text{Confidence} &= \frac{\text{Support}(ABC)}{\text{Support}(AB)} \\ &= \frac{2}{3} \\ &= 66.7\% \end{aligned}$$

If minimum confidence is 60%:

Rule Accepted

## Apriori Pseudocode

L1 = Frequent 1-itemsets

```
for (k = 2; Lk-1 ≠ empty; k++)  
{  
    Generate candidate Ck from Lk-1  
    Prune candidates  
    Count support  
    Generate Lk  
}
```

```
}
```

```
Generate association rules
```

## Advantages of Apriori

- ✓ Easy to understand
- ✓ Reduces search space using pruning
- ✓ Finds strong association rules
- ✓ Widely used in market basket analysis

## Disadvantages of Apriori

- ✗ Multiple database scans required
- ✗ Large number of candidate itemsets
- ✗ Slow on very large datasets
- ✗ High computational cost

## Applications

### Market Basket Analysis

```
Bread → Butter  
Diaper → Beer
```

### E-Commerce

```
Laptop → Mouse
```

### Recommendation Systems

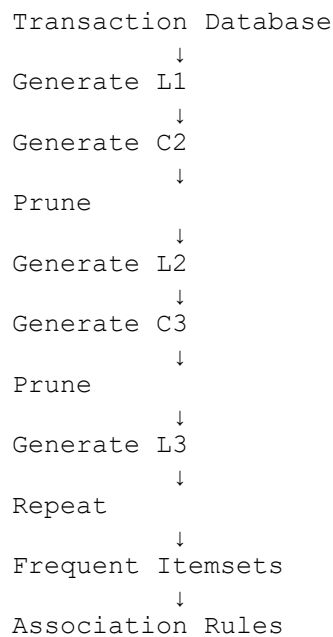
```
Book A → Book B
```

### Web Usage Mining

```
Page A → Page B
```



## Flowchart



## DIC (Dynamic Itemset Counting)

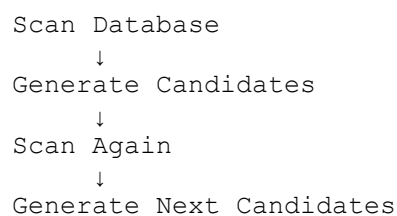
DIC is an improvement over the **Apriori Algorithm** used for **Association Rule Mining**.

### Definition

**Dynamic Itemset Counting (DIC)** is a frequent itemset mining algorithm that dynamically generates and counts candidate itemsets during database scanning instead of waiting for a complete scan as in Apriori.

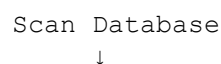
### Main Idea

In Apriori:



Many scans are required.

In DIC:



Count + Generate Candidates Together  
↓  
Less Database Scans

Thus DIC is generally faster than Apriori.

## Why DIC Was Introduced?

### Problem with Apriori

Suppose database contains:

1 Million Transactions

Apriori may require:

Scan 1 → Find L1  
Scan 2 → Find L2  
Scan 3 → Find L3  
Scan 4 → Find L4  
...

Many expensive scans.

### Solution

DIC starts counting larger itemsets before finishing the current scan.

This reduces total scans.

## Basic Concept

The database is divided into blocks.

Example:

1000 Transactions

Block 1 : 250  
Block 2 : 250  
Block 3 : 250  
Block 4 : 250

After each block:

Count Supports  
Generate New Candidates  
Continue Counting

# Four States Used in DIC

This is the most important exam point.

DIC classifies itemsets into 4 states.

## 1. Dashed Circle (DC)

### Meaning

Candidate Itemset  
Not Yet Frequent  
Still Being Counted

Example:

AB

Just generated.

Support is not yet known.

Represented as:

○ (Dashed Circle)

## 2. Dashed Square (DS)

### Meaning

Support  $\geq$  Minimum Support  
Still Being Counted

Itemset appears to be frequent but database scan isn't complete.

Represented as:

□ (Dashed Square)

## 3. Solid Circle (SC)

### Meaning

Database Fully Scanned  
Not Frequent

Support below threshold.

Represented as:

- (Solid Circle)

## 4. Solid Square (SS)

### Meaning

Database Fully Scanned  
Frequent Itemset

Represented as:

- (Solid Square)

## State Transitions

DC  
↓  
DS  
↓  
SS

Frequent itemset path, or

DC  
↓  
SC

Infrequent itemset path.

## Working of DIC

Suppose:

### TID Items

T1 A B

T2 A C

T3 B C

T4 A B C

Minimum Support Count = 2

# Clustering in Data Mining (Detailed)

Clustering is one of the most important topics in **Data Mining** and is a type of **Unsupervised Learning**.

**Clustering is the process of grouping similar data objects into clusters such that objects within the same cluster are highly similar and objects in different clusters are dissimilar.**

In simple words:

Similar Objects → Same Cluster

Different Objects → Different Clusters

## Real-Life Example

Suppose a shopping mall has customer data.

Customers may naturally form groups like:

Cluster 1 → Students

Cluster 2 → Working Professionals

Cluster 3 → Senior Citizens

without anyone labeling them beforehand.

This is clustering.

## Characteristics of Clustering

### Intra-Cluster Similarity

Objects inside the same cluster should be similar.

Cluster A

• • • • •

### Inter-Cluster Dissimilarity

Different clusters should be well separated.

Cluster A

Cluster B

• • •

▲ ▲ ▲

# Applications of Clustering

## 1. Customer Segmentation

Group customers based on purchasing behavior.

## 2. Medical Diagnosis

Group patients with similar symptoms.

## 3. Image Processing

Group similar pixels.

## 4. Document Clustering

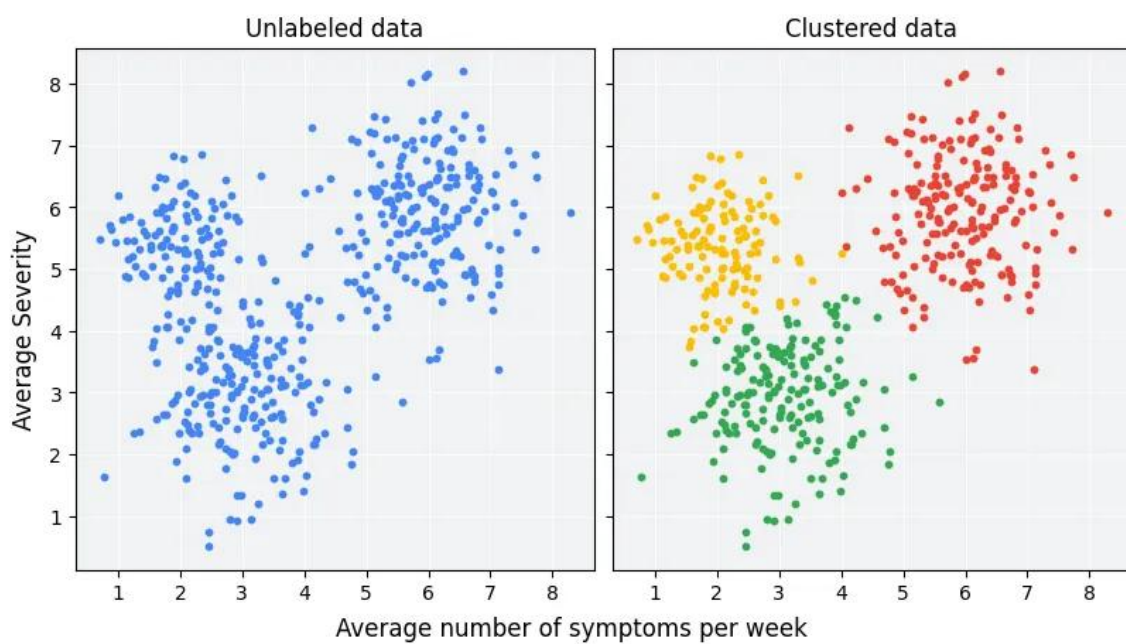
Group similar articles.

## 5. Social Network Analysis

Identify communities of users.

## 6. Market Research

Find customer groups.



# Types of Clustering Methods

There are five major clustering approaches.

## 1. Partitioning Methods

Dataset divided into K clusters.

### Algorithms

- K-Means
- PAM
- CLARA
- CLARANS

Example:

10 Objects  
K = 3

Cluster 1  
Cluster 2  
Cluster 3

### Advantages

- Simple
- Fast

### Disadvantages

- Need K beforehand

## 2. Hierarchical Clustering

Creates a hierarchy of clusters.

### Types

#### Agglomerative (Bottom-Up)

Single Objects  
↓  
Merge  
↓  
Larger Clusters

## Divisive (Top-Down)

One Large Cluster

↓

Split

↓

Smaller Clusters

## Output

Dendrogram

Example:

A    B    C    D

A+B

(A+B) + C

((A+B) + C) + D

## 3. Density-Based Clustering

Clusters are formed based on data density.

### Example Algorithm

- DBSCAN

Idea:

Dense Region = Cluster

Sparse Region = Noise

Example:

••••••

•

Noise Point

### Advantages

- Detects arbitrary-shaped clusters
- Handles noise



## 4. Grid-Based Clustering

Data space divided into grids.

Example:

```
+---+---+---+
|   | • | • |
+---+---+---+
| • | • |   |
+---+---+---+
```

### Example Algorithms

- STING
- CLIQUE

### Advantage

Very fast.

## 5. Model-Based Clustering

Assumes data follows a mathematical model.

Example:

Gaussian Mixture Model (GMM)

Clusters are formed according to probability distributions.

## Popular Clustering Algorithms

### K-Means

#### Steps

1. Choose K centroids.
2. Assign points to nearest centroid.
3. Recalculate centroids.
4. Repeat until convergence.

#### Advantages

- Simple
- Fast

## Disadvantages

- Sensitive to outliers

## PAM

Uses:

Medoids

instead of centroids.

## Advantage

More robust to outliers.

## CLARA

Uses:

Sampling + PAM

for large datasets.

## CLARANS

Uses:

Randomized Search

for very large datasets.

## Features of a Good Cluster

A good clustering result should satisfy the following properties:

### 1. High Intra-Cluster Similarity

Objects within the same cluster should be very similar to each other.

Example:

Cluster A

20

21

22  
23  
24

These values are close to each other, so they form a good cluster.

## 2. Low Inter-Cluster Similarity (High Separation)

Objects belonging to different clusters should be very different.

Example:

| Cluster A | Cluster B |
|-----------|-----------|
| 20 21 22  | 80 81 82  |

The clusters are well separated.

## 3. Compactness

Data points should be closely packed around the cluster center.

Good Cluster

•••••  
•••••

Not:

Poor Cluster

•       •  
     •  
         •

## 4. Separation

Clusters should not overlap significantly.

Good

•••••       ▲▲▲▲▲

instead of

Bad

••▲▲▲▲•

## 5. Meaningfulness

Clusters should represent meaningful groups.

Example:

```
Customer Cluster 1 → Students  
Customer Cluster 2 → Professionals  
Customer Cluster 3 → Retired People
```

The clusters should make practical sense.

## 6. Stability

Small changes in data should not completely change the clustering result.

# PAM (Partitioning Around Medoids)

PAM is a **clustering algorithm** used in Data Mining and Machine Learning.

It is an improvement over **K-Means** because it uses actual data points as cluster centers.

**PAM (Partitioning Around Medoids)** is a partition-based clustering algorithm that divides data into K clusters using representative objects called **medoids**.

### What is a Medoid?

A **medoid** is the most centrally located actual data object in a cluster.

Unlike K-Means:

```
K-Means → Centroid (average point)
```

```
PAM → Medoid (actual data point)
```

## Why PAM?

Problem with K-Means:

```
Outliers affect centroid heavily.
```

Example:

```
1, 2, 3, 4, 100
```

Centroid:

$$\frac{1 + 2 + 3 + 4 + 100}{5} = 22$$

Not representative.

PAM chooses an actual point as center, making it more robust.

## Working of PAM

Suppose:

Data = {2, 4, 5, 7, 10}  
K = 2

### Step 1: Select K Medoids

Choose 2 medoids randomly.

Example:

Medoid1 = 2  
Medoid2 = 7

### Step 2: Assign Points to Nearest Medoid

**Point Nearest Medoid**

|    |   |
|----|---|
| 2  | 2 |
| 4  | 2 |
| 5  | 7 |
| 7  | 7 |
| 10 | 7 |

Clusters:

C1 = {2, 4}  
C2 = {5, 7, 10}

### Step 3: Compute Total Cost

Cost = Sum of distances of points from their medoids.

Example:

Cluster 1

$$\begin{aligned} & |2-2| + |4-2| \\ &= 0+2 \\ &= 2 \end{aligned}$$

Cluster 2

$$\begin{aligned} & |5-7| + |7-7| + |10-7| \\ &= 2+0+3 \\ &= 5 \end{aligned}$$

Total Cost:

7

## Step 4: Swap Medoid with Non-Medoid

Try replacing a medoid.

Example:

Replace 2 with 4

Recalculate cost.

If cost decreases:

Accept Swap

Otherwise:

Reject Swap

## Step 5: Repeat

Continue swapping until no better medoid is found.

Final medoids become cluster centers.

## PAM Algorithm Steps

1. Choose K medoids.
2. Assign each object to nearest medoid.
3. Calculate clustering cost.
4. Swap medoid with non-medoid.
5. Recalculate cost.

6. If cost decreases, keep swap.
7. Repeat until no improvement.

# CLARA (Clustering Large Applications)

## Definition

**CLARA (Clustering Large Applications)** is an extension of the PAM algorithm designed to handle **large datasets efficiently**.

PAM works well for small datasets but becomes very slow for large datasets because it examines all data points. CLARA solves this problem by using **sampling**.

## Problem with PAM

PAM complexity is high because:

```
Every Medoid Swap
    ↓
Checks Entire Dataset
```

For thousands or millions of records:

PAM becomes very slow

## Main Idea of CLARA

Instead of applying PAM to the whole dataset:

```
Large Dataset
    ↓
Take Random Sample
    ↓
Apply PAM
    ↓
Find Best Medoids
    ↓
Use Medoids for Entire Dataset
```

Thus CLARA reduces computation time.

## Working of CLARA

### Step 1: Draw a Sample

Select a random sample from the dataset.

Example:

Dataset = 10,000 objects

Sample = 500 objects

## Step 2: Apply PAM

Run PAM only on the sample.

```
Sample
  ↓
PAM
  ↓
Medoids Found
```

## Step 3: Assign All Objects

Use the obtained medoids to assign every object in the full dataset to a cluster.

## Step 4: Compute Cost

Calculate clustering quality (total distance).

## Step 5: Repeat

Take several different samples and repeat the process.

Example:

Sample 1 → Cost = 2500

Sample 2 → Cost = 2200

Sample 3 → Cost = 2100

Choose the clustering with the lowest cost.

## CLARA Algorithm Steps

1. Select a random sample.
2. Apply PAM on sample.
3. Find medoids.
4. Assign all objects to nearest medoid.
5. Compute clustering cost.
6. Repeat for multiple samples.
7. Select best clustering.

## Example

Suppose:



Dataset = 10000 customers  
K = 3 clusters

### CLARA:

Take 5 samples of 500 customers each

↓  
Run PAM on each sample

↓  
Calculate cost

↓  
Choose best medoids

↓  
Form final clusters

## CLARANS (Clustering Large Applications based upon Randomized Search)

### Definition

**CLARANS** is a partitioning clustering algorithm that combines the advantages of **PAM** and **CLARA** and uses a **randomized search strategy** to find good medoids.

It is designed for **very large datasets**.

## Why CLARANS Was Introduced?

### Problem with PAM

PAM  
↓  
Checks many possible swaps  
↓  
Very slow

### Problem with CLARA

CLARA  
↓  
Uses random samples  
↓  
May miss good medoids

### Solution

### CLARANS:

Uses Random Search  
Instead of Full Search

This improves efficiency while still exploring the dataset well.

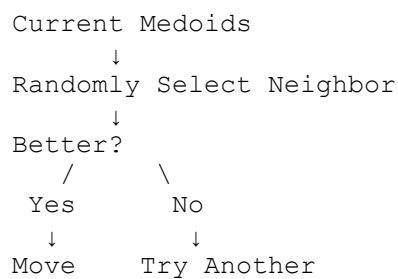
## Main Idea

Think of all possible sets of medoids as nodes in a graph.

Node = One Possible Clustering

CLARANS randomly explores neighboring nodes.

Instead of checking every possible swap:



## Working of CLARANS

### Step 1: Select Initial Medoids

Randomly choose K medoids.

Example:

$K = 2$

Medoids:  
A and D

### Step 2: Generate Neighbor

Create a neighboring solution by:

Replace one medoid  
with one non-medoid

Example:

A, D  
↓  
B, D

## Step 3: Compute Cost

Calculate total clustering cost.

If cost decreases:

Accept New Medoids

Otherwise:

Reject

## Step 4: Repeat Random Search

Continue checking random neighbors.

Current Node

↓  
Neighbor 1  
Neighbor 2  
Neighbor 3  
...

## Step 5: Local Optimum

When no better neighbor is found:

Stop

This gives one local optimum.

## Step 6: Restart

Repeat from another random starting point.

Choose the best clustering found.

## CLARANS Algorithm Steps

1. Randomly choose K medoids.
2. Generate random neighboring solution.
3. Compute clustering cost.
4. If cost decreases, move to neighbor.
5. Repeat until local optimum found.
6. Restart several times.
7. Return best clustering.

# K-Means Clustering

## Definition

**K-Means** is a partitioning clustering algorithm that divides a dataset into **K clusters**, where each cluster is represented by a **centroid** (mean point).

It is one of the most popular **unsupervised learning** algorithms.

## Main Idea

Similar Objects

↓

Same Cluster

Different Objects

↓

Different Clusters

## How K-Means Works

### Step 1: Choose K

Decide the number of clusters.

Example:

$K = 2$

### Step 2: Select Initial Centroids

Choose K points randomly as centroids.

Example:

Data: 2, 4, 6, 8, 10

Centroids:

C1 = 2

C2 = 10

### Step 3: Assign Points to Nearest Centroid

**Point Nearest Centroid**

2      C1

**Point Nearest Centroid**

|    |    |
|----|----|
| 4  | C1 |
| 6  | C1 |
| 8  | C2 |
| 10 | C2 |

Clusters:

Cluster 1 = {2, 4, 6}

Cluster 2 = {8, 10}

**Step 4: Recalculate Centroids**

New centroid = Mean of cluster points.

For Cluster 1:

$$C_1 = \frac{2 + 4 + 6}{3} = 4$$

For Cluster 2:

$$C_2 = \frac{8 + 10}{2} = 9$$

New centroids:

C1 = 4  
C2 = 9

**Step 5: Reassign Points**

Using new centroids:

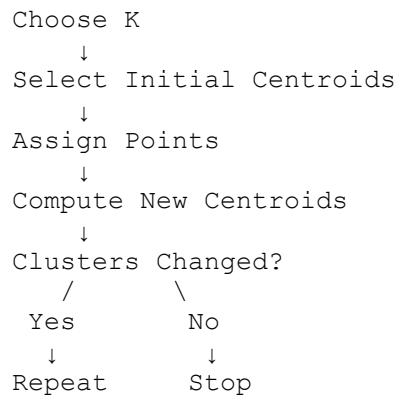
**Point Cluster**

|    |    |
|----|----|
| 2  | C1 |
| 4  | C1 |
| 6  | C1 |
| 8  | C2 |
| 10 | C2 |

No change.

Algorithm stops.

## Flowchart



## Decision Tree (Detailed Explanation)

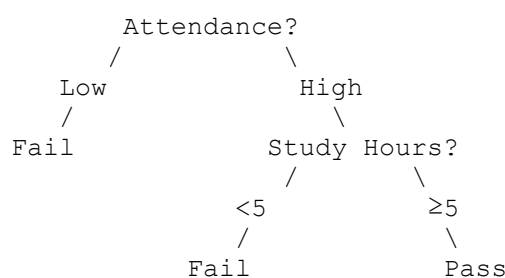
A **Decision Tree** is one of the most important **Supervised Learning** techniques used for **Classification** and **Prediction**.

A **Decision Tree** is a tree-structured model in which internal nodes represent tests on attributes, branches represent outcomes of those tests, and leaf nodes represent class labels or decisions.

It looks like an upside-down tree.

### Example

Suppose we want to predict whether a student will pass.



Here:

- Root Node = Attendance
- Internal Node = Study Hours
- Leaf Nodes = Pass / Fail

# Components of a Decision Tree

## 1. Root Node

Top-most node.

Example:

Attendance?

First attribute used for splitting.

## 2. Internal Node

Represents a test condition.

Example:

Study Hours?

## 3. Branch

Represents outcomes of a test.

Example:

Attendance?

High  
Low

## 4. Leaf Node

Final decision/class.

Example:

Pass  
Fail

# How Decision Tree Works

Decision tree construction is called **Recursive Partitioning**.

## Step 1: Start with Entire Dataset

Example:

| Attendance | Study Hours | Result |
|------------|-------------|--------|
| High       | 8           | Pass   |
| High       | 7           | Pass   |
| Low        | 2           | Fail   |
| Low        | 3           | Fail   |

## Step 2: Choose Best Attribute

Select the attribute that best separates classes.

Common measures:

- Information Gain (ID3)
- Gain Ratio (C4.5)
- Gini Index (CART)

Suppose:

Attendance

gives the best split.

## Step 3: Split Dataset

Attendance  
/            \  
High        Low

## Step 4: Repeat Recursively

For each subset:

Study Hours

may be selected.

Continue splitting.

## Step 5: Stop Splitting

Stop when:

- All records belong to same class.
- No attributes remain.
- Dataset becomes too small.



## Step 6: Create Leaf Nodes

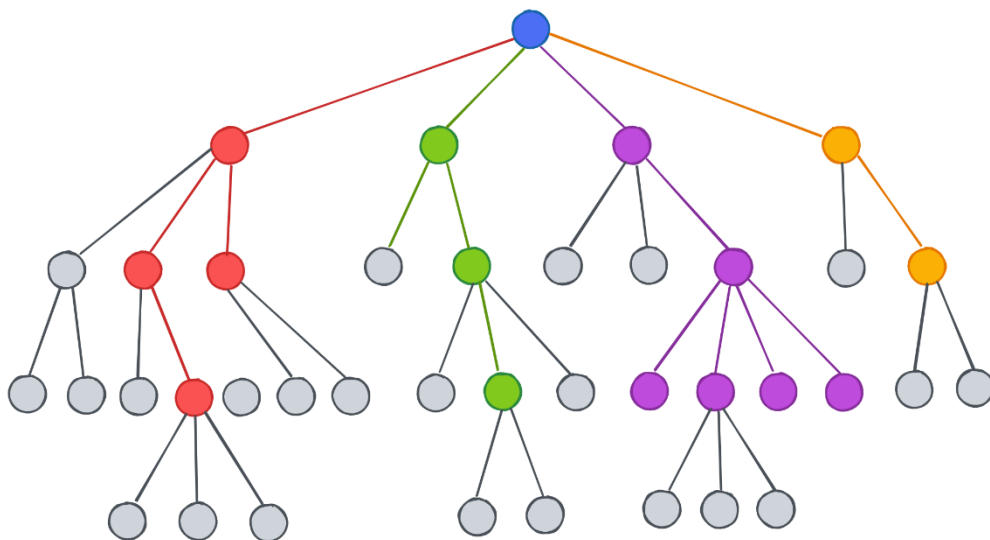
Assign final class.

Pass  
Fail

## Tree Construction Principle

Dataset  
↓  
Choose Best Attribute  
↓  
Split Data  
↓  
Create Child Nodes  
↓  
Repeat  
↓  
Leaf Nodes

This is the standard answer often asked in exams.



# Classification in Data Mining

## Definition

**Classification** is a supervised learning technique used to assign data objects to predefined classes or categories based on a training dataset.

It learns from labeled data and predicts the class label of new data.

## Example

Suppose we have the following training data:

| Study Hours | Result |
|-------------|--------|
| 2           | Fail   |
| 3           | Fail   |
| 7           | Pass   |
| 8           | Pass   |

A classification model learns the relationship and predicts:

```
Study Hours = 6
      ↓
Pass
```

## How Classification Works

### Step 1: Training Phase

Provide labeled data:

```
Input + Class Label
```

Example:

```
Email Features → Spam
Email Features → Not Spam
```

### Step 2: Build Classifier

The algorithm learns patterns from the training data.

Examples:

- Decision Tree
- KNN
- Naive Bayes

- SVM

### Step 3: Testing Phase

Give new unseen data.

```
New Email
      ↓
Classifier
      ↓
Spam / Not Spam
```

## Types of Classification

### Binary Classification

Only two classes.

Examples:

```
Pass / Fail
Spam / Not Spam
Yes / No
```

### Multi-Class Classification

More than two classes.

Examples:

```
Grade A
Grade B
Grade C
Grade D
```

## Common Classification Algorithms

### 1. Decision Tree

Uses a tree structure for classification.

### 2. K-Nearest Neighbor (KNN)

Uses nearest neighbors to classify.

### 3. Naive Bayes

Uses probability theory.

## 4. Support Vector Machine (SVM)

Finds the best separating boundary.

## 5. Random Forest

Uses multiple decision trees.

# Applications

## Spam Detection

Email → Spam / Not Spam

## Medical Diagnosis

Patient → Disease / No Disease

## Loan Approval

Applicant → Approve / Reject

## Fraud Detection

Transaction → Fraud / Genuine

# Classification vs Clustering

| Feature       | Classification | Clustering        |
|---------------|----------------|-------------------|
| Learning Type | Supervised     | Unsupervised      |
| Data          | Labeled        | Unlabeled         |
| Goal          | Predict class  | Discover groups   |
| Example       | Pass/Fail      | Customer Segments |

# K-Nearest Neighbors (KNN)

## Definition

**K-Nearest Neighbors (KNN)** is a supervised learning algorithm used for **classification** and **regression**. It classifies a new data point based on the majority class of its K nearest neighbors.

KNN is called a **lazy learning algorithm** because it does not build a model during training.

## Working Principle

```
Training Data
    ↓
Choose K
    ↓
Calculate Distance
    ↓
Find K Nearest Neighbors
    ↓
Majority Voting
    ↓
Assign Class
```

## Steps of KNN Algorithm

### Step 1: Choose K

Select the number of nearest neighbors.

Example:

$K = 3$

### Step 2: Calculate Distance

Find the distance between the new data point and all training points.

Most commonly used:

#### Euclidean Distance

### Step 3: Find K Nearest Neighbors

Choose the K points having the smallest distances.

### Step 4: Majority Voting

For classification:

Most Frequent Class  
↓  
Final Prediction

## Example

Training Data:

### Marks Result

|    |      |
|----|------|
| 40 | Fail |
| 45 | Fail |
| 60 | Pass |
| 70 | Pass |

New Student:

Marks = 55

Choose:

K = 3

### Distance Calculation

#### Marks Result Distance from 55

|    |      |    |
|----|------|----|
| 40 | Fail | 15 |
| 45 | Fail | 10 |
| 60 | Pass | 5  |
| 70 | Pass | 15 |

Nearest 3 neighbors:

60 → Pass  
45 → Fail  
40 → Fail

Voting:

Pass = 1  
Fail = 2

Prediction:

Fail

# Choosing the Value of K

## Small K

$$K = 1$$

- Sensitive to noise
- May overfit

## Large K

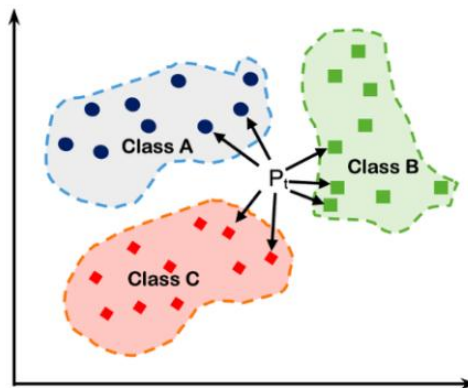
$$K = 15$$

- More stable
- May underfit

Usually:

$$K = 3, 5, 7$$

Use odd values to avoid ties.



## K-Means vs KNN

| Feature         | K-Means                    | KNN (K-Nearest Neighbors)           |
|-----------------|----------------------------|-------------------------------------|
| Type            | Clustering Algorithm       | Classification/Regression Algorithm |
| Learning        | Unsupervised Learning      | Supervised Learning                 |
| Data            | Unlabeled Data             | Labeled Data                        |
| Purpose         | Group similar objects      | Predict class/value                 |
| Output          | Clusters                   | Class Label or Numeric Value        |
| Uses Centroids? | Yes                        | No                                  |
| Training Phase  | Iteratively forms clusters | Stores training data                |
| Examples        | Customer Segmentation      | Spam Detection                      |

# Naive Bayes Classifier

## Definition

**Naive Bayes** is a supervised machine learning algorithm used for **classification** that is based on **Bayes' Theorem** and assumes that all attributes are independent of each other.

The word "**Naive**" comes from the assumption that features are independent.

## Bayes' Theorem

Naive Bayes works using:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}$$

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)} = \frac{0.85(0.20)}{0.25} = 0.68$$

$$\frac{P(A)}{P(B | A) + P(B | \neg A)}$$

$P(B)=0.25$  $P(B|A)P(A)=0.17$  $P(A|B)\sim 0.68$ Posterior = useful evidence / total evidence

Where:

- $P(A | B)$ = Posterior Probability
- $P(B | A)$ = Likelihood
- $P(A)$ = Prior Probability
- $P(B)$ = Evidence

## Why is it Called "Naive"?

Suppose we want to classify a fruit.

Features:

Color = Red  
Shape = Round

Naive Bayes assumes:

Color and Shape are independent

which may not always be true in real life.



# Working of Naive Bayes

## Step 1: Calculate Prior Probability

Example:

| Class    | Count |
|----------|-------|
| Spam     | 40    |
| Not Spam | 60    |

Total Emails = 100

$$P(\text{Spam}) = 40/100 = 0.4$$

$$P(\text{Not Spam}) = 60/100 = 0.6$$

## Step 2: Calculate Likelihood

Find probability of features given the class.

Example:

$$P(\text{Free} \mid \text{Spam})$$

Probability that the word "Free" appears in spam emails.

## Step 3: Apply Bayes Theorem

Compute:

$$P(\text{Spam} \mid \text{Free})$$

and

$$P(\text{Not Spam} \mid \text{Free})$$

## Step 4: Choose Higher Probability

Higher Probability

↓  
Predicted Class

## Example

Suppose:

| Email          | Class    |
|----------------|----------|
| Free Offer     | Spam     |
| Win Money      | Spam     |
| Meeting Today  | Not Spam |
| Project Update | Not Spam |

New Email:

"Free Money"

Naive Bayes calculates:

$P(\text{Spam} \mid \text{Free Money})$

$P(\text{Not Spam} \mid \text{Free Money})$

If:

$P(\text{Spam} \mid \text{Free Money})$

>

$P(\text{Not Spam} \mid \text{Free Money})$

Prediction:

Spam

## Types of Naive Bayes

### 1. Gaussian Naive Bayes

Used for continuous numerical data.

Example:

Age  
Salary  
Height

### 2. Multinomial Naive Bayes

Used for text classification.

Example:

Spam Detection  
Document Classification

### 3. Bernoulli Naive Bayes

Uses binary features.

Example:

Word Present = Yes/No

## Support Vector Machine (SVM)

**Support Vector Machine (SVM)** is a supervised machine learning algorithm used for **classification** and **regression**. It works by finding the **best boundary (hyperplane)** that separates different classes with the maximum margin.

### Basic Idea

Suppose we have two classes:

o o o o o  
x x x x x

Many lines can separate them:

Line A  
Line B  
Line C

SVM chooses the **best line** that leaves the **largest distance (margin)** from both classes.

o o o o o  
=====  
Best Hyperplane  
=====  
x x x x x

## Important Terms

### 1. Hyperplane

A decision boundary that separates classes.

In 2D:

A straight line

In 3D:

A plane

In higher dimensions:

Hyperplane

## 2. Margin

Distance between the hyperplane and nearest data points.

Class A

○ ○  
○ ○

----- Margin -----  
==== Hyperplane ====

× ×  
× ×

Class B

### Rule

Larger Margin

↓  
Better Classification

## 3. Support Vectors

The data points closest to the hyperplane.

○ ○ ●

=====

▲ × ×

Here:

● and ▲

are support vectors.

They determine the position of the hyperplane.

# Working of SVM

## Step 1

Take labeled training data.

## Step 2

Find possible separating hyperplanes.

## Step 3

Calculate margins.

## Step 4

Choose hyperplane with maximum margin.

## Step 5

Use it to classify new data.

## Flow

```
Training Data
  ↓
Find Hyperplanes
  ↓
Compute Margins
  ↓
Select Maximum Margin
  ↓
Classification Model
```

## Example

Suppose we want to classify:

Pass  
Fail

Training Data:

### Hours Studied Result

|   |      |
|---|------|
| 2 | Fail |
| 3 | Fail |
| 8 | Pass |
| 9 | Pass |

SVM finds the optimal boundary.

New Student:

Hours = 7

Prediction:

Pass

## Types of SVM

### 1. Linear SVM

Used when classes can be separated by a straight line.

○ ○ ○  
-----  
× × ×

### 2. Non-Linear SVM

Used when data is not linearly separable.

○ ○ ○  
    × ×  
○ ○ ○

Uses **Kernel Functions**.

## Kernel Trick

Kernel transforms data into a higher dimension where separation becomes easier.

### Common Kernels

#### Linear Kernel

Simple straight boundary

#### Polynomial Kernel

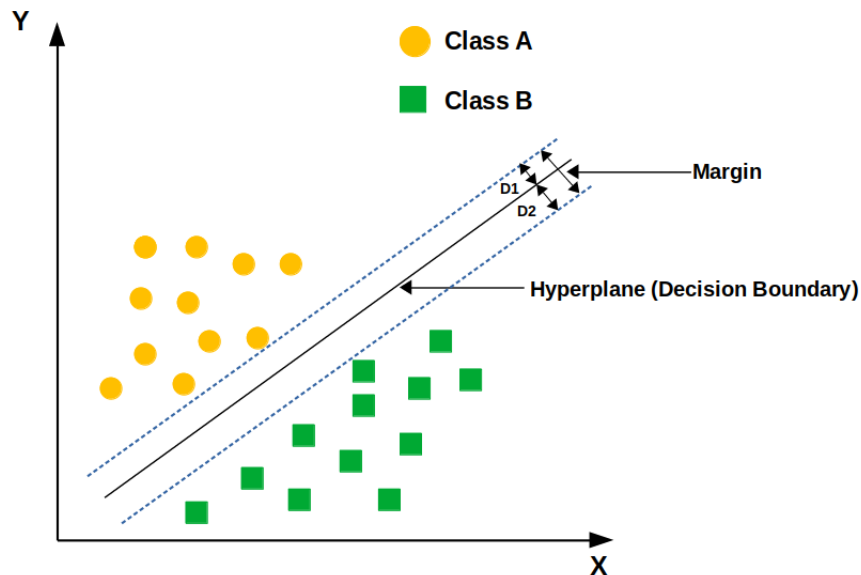
Curved boundary

#### RBF (Radial Basis Function)

Most commonly used.

### Sigmoid Kernel

Neural-network-like behavior.



## Random Forest

### Definition

**Random Forest** is a supervised machine learning algorithm that combines multiple Decision Trees and makes predictions based on majority voting (classification) or averaging (regression).

It is an **Ensemble Learning** technique.

### Why is it Called Random Forest?

- **Forest** = Collection of many Decision Trees.
- **Random** = Each tree is trained on a random subset of data and features.

```
Tree 1
Tree 2
Tree 3
Tree 4
Tree 5
↓
Random Forest
```

## Basic Idea

Suppose we want to predict whether a loan should be approved.

Individual trees predict:

Tree 1 → Yes  
Tree 2 → Yes  
Tree 3 → No  
Tree 4 → Yes  
Tree 5 → No

Voting:

Yes = 3  
No = 2

Final Prediction:

Yes

This is called **Majority Voting**.

## Working of Random Forest

### Step 1: Create Bootstrap Samples

Randomly select records from the dataset (with replacement).

Example:

Original Data:

A B C D E

Sample 1:

A B B D E

Sample 2:

A C D D E

### Step 2: Build Multiple Decision Trees

Each sample creates a separate decision tree.



Dataset



Tree 1

Dataset



Tree 2

Dataset



Tree 3

## Step 3: Random Feature Selection

Instead of considering all attributes, each tree uses a random subset of features.

Example:

Features:

Age  
Salary  
Experience  
Education

Tree 1 may use:

Age, Salary

Tree 2 may use:

Salary, Education

## Step 4: Voting

For classification:

All Trees



Majority Vote



Final Class

For regression:

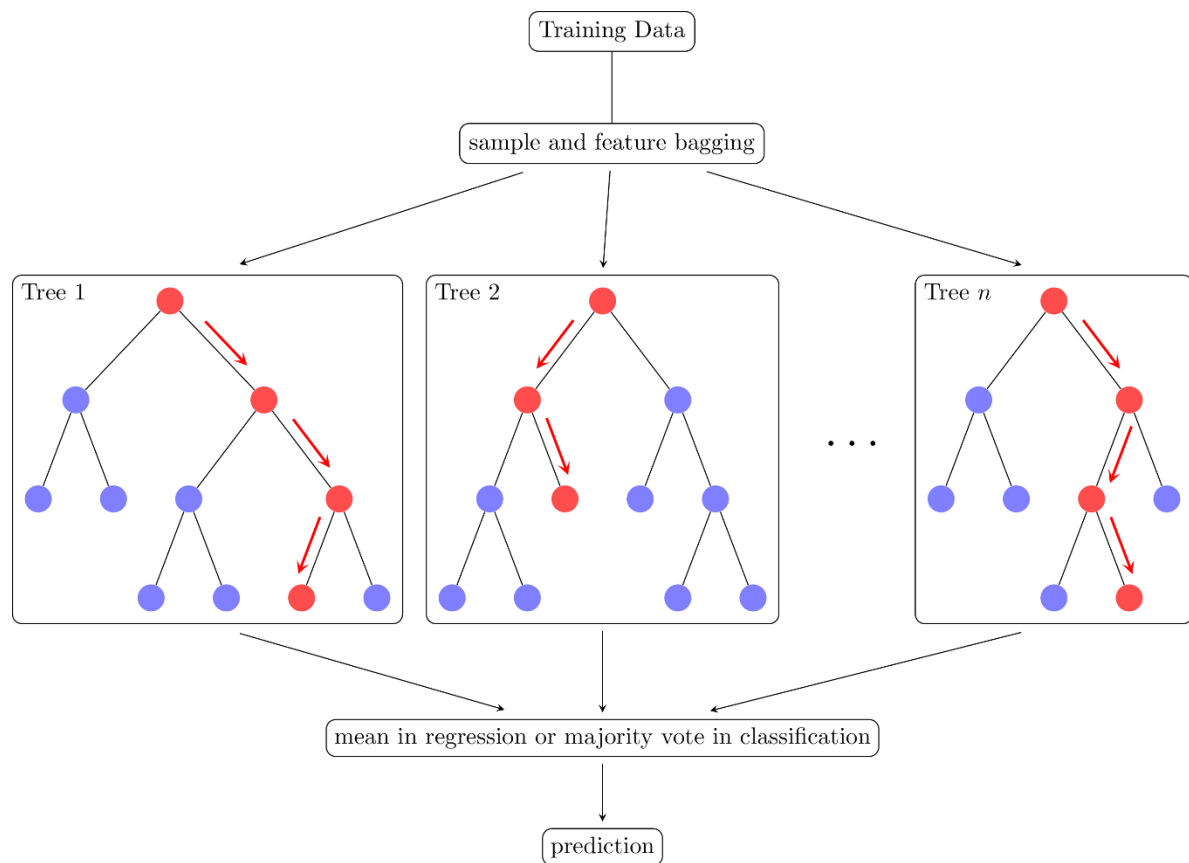
All Trees



Average Output



Final Value



## Flow Diagram

```

Training Data
  ↓
Bootstrap Sampling
  ↓
Build Many Trees
  ↓
Random Features
  ↓
Predictions from Trees
  ↓
Majority Voting
  ↓
Final Prediction
  
```

## Example

Predict:

Pass / Fail

Results from trees:

```

Tree 1 → Pass
Tree 2 → Pass
Tree 3 → Fail
  
```

Tree 4 → Pass  
Tree 5 → Pass

Voting:

Pass = 4  
Fail = 1

Final Output:

Pass

## Why Random Forest is Better than a Single Decision Tree

Decision Tree:

One Expert

Random Forest:

Many Experts

Multiple trees reduce mistakes and overfitting.

## FP-Tree (Frequent Pattern Tree)

FP-Tree is one of the most important topics in Data Mining and is often asked as:

- Explain FP-Tree.
- FP-Growth Algorithm.
- Construction of FP-Tree.
- Advantages of FP-Growth over Apriori.

## What is FP-Tree?

**FP-Tree (Frequent Pattern Tree)** is a compact tree structure used to store transaction data and discover frequent itemsets without generating candidate itemsets.

It is used in the **FP-Growth Algorithm**.

# Why FP-Tree?

## Problem with Apriori

Apriori:

```
Generate Candidates
  ↓
Scan Database
  ↓
Generate More Candidates
  ↓
Scan Again
```

Problems:

- ✗ Too many candidate itemsets
- ✗ Multiple database scans
- ✗ Slow for large datasets

## FP-Growth Solution

```
Scan Database Twice
  ↓
Build FP-Tree
  ↓
Mine Frequent Patterns
```

Advantages:

- ✓ No candidate generation
- ✓ Faster
- ✓ Compact storage

## Example Dataset

| TID | Items   |
|-----|---------|
| T1  | A,B,D,E |
| T2  | B,C,E   |
| T3  | A,B,C,E |
| T4  | B,E     |
| T5  | A,B,C,E |

Minimum Support = 2

## Step 1: Count Item Frequencies

Count occurrences.

### Item Count

|   |   |
|---|---|
| A | 3 |
| B | 5 |
| C | 3 |
| D | 1 |
| E | 5 |

Since minimum support = 2

Remove:

D

because support = 1

Remaining:

### Item Support

|   |   |
|---|---|
| B | 5 |
| E | 5 |
| A | 3 |
| C | 3 |

## Step 2: Sort Frequent Items

Arrange descending support.

B > E > A > C

## Step 3: Reorder Transactions

T1

Original:

A B D E

Remove D

A B E

Reorder:

B E A

**T2**

B C E

Reorder:

B E C

**T3**

A B C E

Reorder:

B E A C

**T4**

B E

**T5**

A B C E

Reorder:

B E A C

## Step 4: Construct FP-Tree

**Start with:**

NULL

**Insert T1:**

```
NULL
|
B:1
|
E:1
|
A:1
```

**Insert T2:**

```

NULL
 |
B:2
 |
E:2
 / \
A:1 C:1

```

**Insert T3:**

```

NULL
 |
B:3
 |
E:3
 / \
A:2 C:1
 |
C:1

```

**Insert T4:**

```

NULL
 |
B:4
 |
E:4
 / \
A:2 C:1
 |
C:1

```

**Insert T5:**

```

NULL
 |
B:5
 |
E:5
 / \
A:3 C:1
 |
C:2

```

**Final FP Tree:**

```

      NULL
      |
      B:5
      |
      E:5
     /  \
    A:3   C:1
     |
    C:2

```

**Header Table**

FP-Tree maintains a Header Table.

**Item Support**

|   |   |
|---|---|
| B | 5 |
| E | 5 |
| A | 3 |
| C | 3 |

Each item points to its occurrences in the tree.

Purpose:

Quick Traversal

## Mining Frequent Patterns

Start from least frequent item.

Order:

C  
A  
E  
B

Create:

**Conditional Pattern Base**

and

**Conditional FP Tree**

for each item.

Then generate frequent patterns.

Example patterns:

{B}

{E}

{B, E}

{A, B}

{A, E}

{A, B, E}



{B,C}

{B,E,C}

## FP-Growth Algorithm Steps

```

Scan Database
  ↓
Find Frequent Items
  ↓
Sort Items
  ↓
Build FP Tree
  ↓
Generate Conditional Pattern Base
  ↓
Generate Conditional FP Tree
  ↓
Frequent Itemsets
  
```

## FP-Tree vs Apriori

| Feature              | Apriori | FP-Growth |
|----------------------|---------|-----------|
| Candidate Generation | Yes     | No        |
| Database Scans       | Many    | 2         |
| Speed                | Slower  | Faster    |
| Memory Usage         | Higher  | Lower     |
| Large Dataset        | Poor    | Better    |

## Pearson's Correlation Coefficient

### Definition

**Pearson's Correlation Coefficient (r)** is a statistical measure that shows the **strength and direction of the linear relationship** between two variables.

It is widely used in **Data Mining, Statistics, Machine Learning, and Data Analysis**.

### Formula

The Pearson correlation coefficient is:

$$r = \frac{\sum(x - \bar{x})(y - \bar{y})}{\sqrt{\sum(x - \bar{x})^2 \sum(y - \bar{y})^2}}$$

Where:

- $x, y$  = observations
- $\bar{x}$  = mean of X
- $\bar{y}$  = mean of Y
- $r$  = correlation coefficient

## Range of r

$$-1 \leq r \leq +1$$

### Interpretation

| Value of r | Meaning                      |
|------------|------------------------------|
| +1         | Perfect Positive Correlation |
| 0          | No Correlation               |
| -1         | Perfect Negative Correlation |

## Types of Correlation

### 1. Positive Correlation

When one variable increases, the other also increases.

Example:

Study Hours  $\uparrow$   
Marks  $\uparrow$   
 $r > 0$

### 2. Negative Correlation

When one variable increases, the other decreases.

Example:

Speed  $\uparrow$   
Travel Time  $\downarrow$   
 $r < 0$

### 3. Zero Correlation

No relationship between variables.

Example:

Shoe Size  
Exam Marks  
 $r \approx 0$

## Example

Consider:

**Study Hours (X) Marks (Y)**

|   |    |
|---|----|
| 1 | 40 |
| 2 | 50 |
| 3 | 60 |
| 4 | 70 |
| 5 | 80 |

Observation:

Study Hours ↑  
Marks ↑

This gives:

$r = +1$

Perfect positive correlation.

## Interpretation Table

| r Value        | Interpretation       |
|----------------|----------------------|
| 0.90 to 1.00   | Very Strong Positive |
| 0.70 to 0.89   | Strong Positive      |
| 0.40 to 0.69   | Moderate Positive    |
| 0.10 to 0.39   | Weak Positive        |
| 0              | No Correlation       |
| -0.10 to -0.39 | Weak Negative        |
| -0.40 to -0.69 | Moderate Negative    |
| -0.70 to -0.89 | Strong Negative      |
| -0.90 to -1.00 | Very Strong Negative |

## Applications

-  **Market Analysis** Advertising Cost ↔ Sales
-  **Data Mining:** Attribute Relationship Analysis
-  **Healthcare:** Weight ↔ Blood Pressure
-  **Finance:** Risk ↔ Return
-  **Machine Learning** Feature Selection: Highly correlated features can sometimes be removed.

# Time Series Data (Detailed Explanation)

Time Series is an important Data Mining and Machine Learning topic, especially for **forecasting future values**.

**Time Series Data** is a collection of observations recorded at successive points in time, usually at equal intervals.

Examples:

| Day       | Temperature |
|-----------|-------------|
| Monday    | 30°C        |
| Tuesday   | 32°C        |
| Wednesday | 31°C        |
| Thursday  | 35°C        |

Here, the data depends on **time**, so it is a time series.

## Examples of Time Series Data

### Stock Prices

Day 1 → ₹100  
Day 2 → ₹105  
Day 3 → ₹102

### Weather Data

Daily Temperature  
Monthly Rainfall

### Sales Data

Jan → ₹10,000  
Feb → ₹12,000  
Mar → ₹15,000

### Website Traffic

Hourly Visitors

### Electricity Consumption

Daily Power Usage

# Characteristics of Time Series Data

Unlike normal datasets:

Age → Salary

the order matters.

Example:

100  
120  
140  
160

and

160  
120  
100  
140

contain the same values but represent different time series.

## Important Point

Time Order Matters

# Components of Time Series

A time series is usually made up of four components.

## 1. Trend (T)

Long-term movement of data.

Example:

Company Sales

2021 → 100  
2022 → 120  
2023 → 140  
2024 → 170

Trend:

Increasing

## 2. Seasonal Variation (S)

Repeating patterns over fixed periods.

Example:

Ice Cream Sales

Higher in summer, lower in winter.

Summer ↑  
Winter ↓

Occurs every year.

## 3. Cyclical Variation (C)

Long-term fluctuations.

Example:

Economic Boom  
↓  
Economic Recession  
↓  
Boom

Duration is not fixed.

## 4. Irregular Variation (I)

Random unpredictable changes.

Example:

Pandemic  
Earthquake  
War  
Flood

Cannot be predicted easily.

## Time Series Model

Time Series is often represented as:

Time Series  
=  
Trend  
+  
Seasonal

+  
Cyclical  
+  
Irregular

Memory Trick:

TSCI

Trend  
Seasonal  
Cyclical  
Irregular

## Types of Time Series

### 1. Univariate Time Series

Only one variable changes over time.

Example:

Daily Temperature

#### **Day Temp**

1 30  
2 32  
3 31

### 2. Multivariate Time Series

Multiple variables change over time.

Example:

| Day | Temp | Humidity |
|-----|------|----------|
| 1   | 30   | 70       |
| 2   | 32   | 75       |
| 3   | 31   | 72       |

## Time Series Analysis

Time Series Analysis is used to:

### **Understand Past Behavior**

Sales History

## Detect Patterns

Trend  
Seasonality

## Forecast Future Values

Future Sales  
Future Temperature  
Future Stock Price

# Forecasting Methods

## 1. Moving Average

Uses average of recent observations.

Example:

10, 12, 14

Moving Average:

$$(10+12+14)/3 = 12$$

## 2. Exponential Smoothing

Gives more importance to recent data.

## 3. ARIMA

Most famous forecasting model.

Full Form:

Auto Regressive Integrated Moving Average

Used for:

Sales Forecasting  
Weather Forecasting  
Financial Forecasting



# Methods for Computing Best Split:

The best splitting attribute in a decision tree can be selected using Information Gain, Gain Ratio, and Gini Index. Information Gain uses entropy to measure the reduction in uncertainty after a split and selects the attribute with the highest gain. Gain Ratio improves Information Gain by normalizing it with split information and reduces bias toward attributes with many values. Gini Index measures impurity and selects the attribute with the lowest Gini value. These measures help determine the most effective attribute for partitioning the data.

The three most common methods are:

1. **Information Gain**
2. **Gain Ratio**
3. **Gini Index**

## 1. Information Gain (Entropy-Based)

Used in **ID3 Decision Tree**.

It measures how much uncertainty is reduced after splitting.

### Entropy

Entropy measures impurity or disorder in a dataset.

### Properties

Entropy = 0  
→ Pure dataset (all same class)

Entropy = 1  
→ Maximum impurity (50%-50%)

### Information Gain

Information Gain  
=  
Entropy before split  
-  
Entropy after split

Higher Information Gain = Better Attribute

### Example

Suppose:

Dataset Entropy = 1

After splitting by Outlook:

Entropy = 0.3

Then:

Information Gain = 1 - 0.3 = 0.7

Since gain is high, Outlook is a good splitting attribute.

## 2. Gain Ratio

Used in **C4.5 Decision Tree**.

Problem with Information Gain:

It favors attributes having many distinct values.

Example:

Student\_ID

Every student has a unique ID.

Information Gain may wrongly choose it.

### Gain Ratio Formula

$$\text{Gain Ratio} = \frac{\text{Information Gain}}{\text{Split Information}}$$

### Split Information

Measures how broadly the data is divided.

Gain Ratio corrects the bias of Information Gain.

### Rule

Higher Gain Ratio  
=  
Better Split

### 3. Gini Index

Used in **CART (Classification and Regression Trees)**.

Measures impurity directly.

#### Properties

Gini = 0  
→ Pure Node

Higher Gini  
→ More Impure

#### Rule

Choose the split with:

Lowest Gini Index

## How CLARA and CLARANS are Different:

### CLARA (Clustering Large Applications)

#### Idea

Instead of using the whole dataset, CLARA takes a **sample** of the dataset and runs PAM on that sample.

#### Example

Suppose:

Dataset = 1,000,000 records

CLARA chooses:

Sample = 5000 records

Then:

Run PAM on sample

Find medoids and use them for the whole dataset.

#### Problem

If the sample is bad:

Sample misses important clusters

then clustering quality becomes poor.

## CLARANS (Clustering Large Applications based upon Randomized Search)

### Idea

Instead of fixed samples, CLARANS performs a **random search** in the medoid space.

It randomly examines neighboring solutions.

### Example

Suppose current medoids are:

M1 = A

M2 = B

CLARANS randomly tries:

M1 = A

M2 = C

If cost decreases:

Accept new medoids

Otherwise:

Reject

Continue random exploration.

# Spatial & Temporal Data Mining

## 1. Spatial Data Mining

**Spatial Data Mining** is the process of discovering useful patterns, relationships, and knowledge from **spatial (location-based) data**.

### Definition

Spatial Data Mining refers to the extraction of interesting and useful patterns from spatial databases such as maps, satellite images, GIS data, and geographical information.

### Spatial Data Contains

Location  
Coordinates  
Distance  
Direction  
Area  
Region

### Examples

- Finding crime-prone areas in a city.
- Identifying regions with high rainfall.
- Discovering disease outbreak zones.
- Traffic pattern analysis.

### Example

Kolkata → High Population  
Howrah → Medium Population  
Durgapur → Low Population

Spatial mining finds geographic patterns among these regions.

### Applications

- GIS (Geographic Information Systems)
- Urban Planning
- Weather Forecasting
- Disaster Management
- Navigation Systems

## 2. Temporal Data Mining

**Temporal Data Mining** is the process of discovering patterns from data that changes over time.

### Definition

Temporal Data Mining extracts useful patterns, trends, and relationships from time-oriented data.

### Temporal Data Contains

Date  
Time  
Month  
Year  
Timestamp

### Examples

#### Sales Data

##### Month Sales

|     |     |
|-----|-----|
| Jan | 100 |
| Feb | 120 |
| Mar | 140 |

Temporal mining finds:

Sales increasing over time

#### Stock Market

Monday → ₹100  
Tuesday → ₹110  
Wednesday → ₹105

Temporal mining identifies trends and predicts future prices.

### Applications

- Stock Market Analysis
- Weather Forecasting
- Sales Forecasting
- Healthcare Monitoring
- Time Series Analysis

## Difference Between Spatial and Temporal Data Mining

| Spatial Data Mining             | Temporal Data Mining              |
|---------------------------------|-----------------------------------|
| Deals with location-based data  | Deals with time-based data        |
| Focuses on space and geography  | Focuses on time and sequence      |
| Uses coordinates, maps, regions | Uses dates, timestamps, intervals |
| Example: Finding crime hotspots | Example: Predicting stock prices  |

## Splitting Attribute and Splitting Criterion

These concepts are used in **Decision Tree Induction**.

### 1. Splitting Attribute

A **splitting attribute** is the attribute selected to divide the dataset into smaller subsets during the construction of a decision tree.

#### Definition

A splitting attribute is the attribute used to partition the data at a decision node.

#### Example

Dataset:

##### Outlook Humidity Play

|       |        |     |
|-------|--------|-----|
| Sunny | High   | No  |
| Sunny | Normal | Yes |
| Rain  | High   | Yes |

Suppose **Outlook** is chosen first.

Decision Tree:

```
      Outlook
     /  |  \
    /   |   \
Sunny Rain Overcast
```

Here:

Outlook

is the **splitting attribute**.

## 2. Splitting Criterion

A **splitting criterion** is the method used to decide which attribute should be selected as the splitting attribute.

### Definition

A splitting criterion is a measure used to evaluate and choose the best attribute for splitting the data.

### Common Splitting Criteria

#### (a) Information Gain

- Used in **ID3**
- Based on entropy
- Choose attribute with highest Information Gain

Higher Gain → Better Split

#### (b) Gain Ratio

- Used in **C4.5**
- Improves Information Gain
- Removes bias toward attributes with many values

Higher Gain Ratio → Better Split

#### (c) Gini Index

- Used in **CART**
- Measures impurity

Lower Gini → Better Split



# ROLAP, Molap & HOLAP

## What is OLAP?

OLAP allows users to perform:

- Roll-up
- Drill-down
- Slice
- Dice
- Pivot

on warehouse data for decision making.

There are **3 main OLAP models**:

1. ROLAP → Relational OLAP
2. MOLAP → Multidimensional OLAP
3. HOLAP → Hybrid OLAP

## 1. ROLAP (Relational OLAP)

### Definition

ROLAP stores data in relational databases (tables) and performs analysis using SQL queries.

### Architecture

```
Data Warehouse
    ↓
Relational Tables
    ↓
SQL Queries
    ↓
Reports/Analysis
```

### Example

Sales Table:

| Product | Region  | Sales |
|---------|---------|-------|
| Laptop  | Kolkata | 10000 |
| Mobile  | Delhi   | 8000  |

To find total sales:

```
SELECT SUM(Sales)
FROM Sales;
```

ROLAP executes SQL on relational tables.

### Advantages

- ✓ Can handle huge datasets
- ✓ Uses existing relational databases
- ✓ Less storage overhead

### Disadvantages

- ✗ Query processing may be slower
- ✗ Complex aggregations take time

## 2. MOLAP (Multidimensional OLAP)

### Definition

MOLAP stores data in multidimensional cubes rather than relational tables.

### Architecture

```
Data Warehouse
    ↓
OLAP Cube
    ↓
Precomputed Aggregates
    ↓
Analysis
```

### Example

A sales cube:

```
      Time
      |
      |
Product ----+---- Region
```

Dimensions:

Product  
Region  
Time

Measure:

Sales

Instead of calculating totals every time, MOLAP stores many results beforehand.

### Example Query

Total Mobile Sales in Kolkata during 2025

Answer is retrieved directly from the cube.

### Advantages

- ✓ Very fast query response
- ✓ Excellent for complex analysis
- ✓ Supports multidimensional views naturally

### Disadvantages

- ✗ Cube creation requires extra storage
- ✗ Poor scalability for very large datasets

## 3. HOLAP (Hybrid OLAP)

### Definition

HOLAP combines the features of both ROLAP and MOLAP.

### Architecture

Detailed Data  
↓  
Stored in Relational Tables (ROLAP)

Summary Data  
↓  
Stored in Cubes (MOLAP)

### Example

Suppose a company has:

100 million sales records

HOLAP stores:

Detailed transactions  
→ Relational tables

Monthly/Yearly summaries  
→ OLAP cubes

When a user asks:

Total yearly sales

HOLAP uses the cube.

When the user asks:

Show every transaction of 15th June

HOLAP uses relational tables.

### Advantages

- ✓ Faster than ROLAP
- ✓ Less storage than MOLAP
- ✓ Supports detailed and summary analysis

### Disadvantages

- ✗ More complex architecture
- ✗ More difficult to maintain

### Comparison Table

| Feature             | ROLAP             | MOLAP                 | HOLAP          |
|---------------------|-------------------|-----------------------|----------------|
| Full Form           | Relational OLAP   | Multidimensional OLAP | Hybrid OLAP    |
| Storage             | Relational Tables | Cubes                 | Tables + Cubes |
| Query Speed         | Slowest           | Fastest               | Medium/Fast    |
| Scalability         | Excellent         | Limited               | Good           |
| Storage Requirement | Low               | High                  | Medium         |
| Detail Data Access  | Excellent         | Limited               | Excellent      |
| Complexity          | Low               | Medium                | High           |

# Tree Pruning

In a Decision Tree, sometimes the tree becomes **too large and complex**.

It starts memorizing the training data instead of learning general patterns. This is called **overfitting**.

## Definition

Tree pruning is the process of removing unnecessary branches from a decision tree to reduce overfitting and improve prediction accuracy.

## Why Pruning is Needed?

Without pruning:

```
Large Tree
  ↓
Overfitting
  ↓
Poor performance on new data
```

With pruning:

```
Smaller Tree
  ↓
Better Generalization
  ↓
Higher Accuracy on unseen data
```

## Example

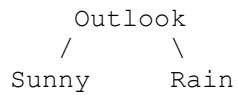
Suppose we have:

```
      Outlook
     /      \
  Sunny     Rain
   |
 Humidity
   |
Wind Speed
   |
Temperature
```

The lower branches may classify only 1–2 training examples.

These branches may not be useful.

Pruning removes them:



Result:

- Simpler tree
- Faster classification
- Less overfitting

## Techniques of Tree Pruning

There are **two main techniques**:

### 1. Pre-Pruning (Early Stopping)

Also called:

Forward Pruning

#### Idea

Stop growing the tree before it becomes too complex.

#### Conditions for Stopping

- Minimum number of records reached
- Maximum tree depth reached
- Information gain becomes very small
- Node becomes sufficiently pure

#### Example

Suppose:

Information Gain = 0.001

Threshold:

Minimum Gain = 0.01

Since:

$0.001 < 0.01$

Do not split further.

#### Advantages

✓ Faster

✓ Small tree

## Disadvantages

✗ May stop too early

✗ Can miss useful patterns

## 2. Post-Pruning

Also called:

Backward Pruning

### Idea

1. Build the complete tree.
2. Remove unnecessary branches afterward.

### Process

```
Build Full Tree
    ↓
Evaluate Branches
    ↓
Remove Weak Branches
    ↓
Get Simplified Tree
```

### Example

Original Tree:

```
Outlook
├── Sunny
│   ├── Humidity
│   └── Wind
└── Rain
```

Suppose "Wind" contributes very little.

After pruning:

```
Outlook
├── Sunny
│   └── Humidity
└── Rain
```

### Advantages

- ✓ Usually more accurate
- ✓ Reduces overfitting effectively

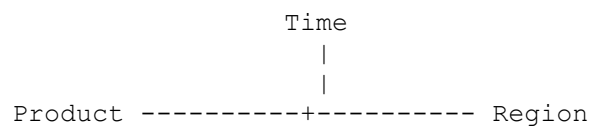
### Disadvantages

- ✗ More computational cost
- ✗ Requires building full tree first

## Slicing Dicing roll up and Drill down

These are the **four most important OLAP operations** used in Data Warehousing.

Let's use a simple Sales Cube:

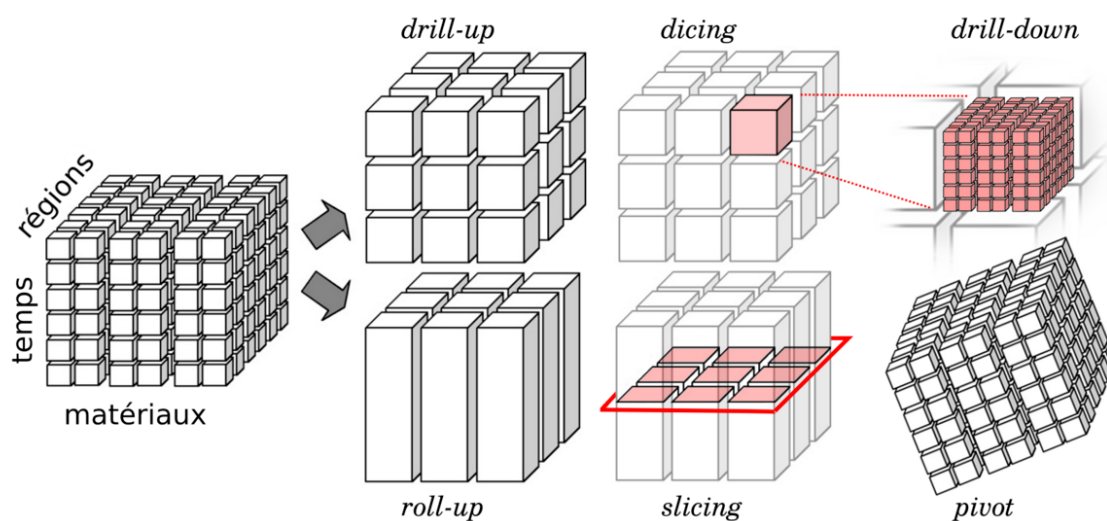


Dimensions:

- Product
- Region
- Time

Measure:

- Sales





# 1. Roll-Up (Aggregation)

## Definition

Roll-up summarizes data by moving from a lower level to a higher level in a hierarchy.

## Example

Sales by City:

| City     | Sales |
|----------|-------|
| Kolkata  | 100   |
| Howrah   | 50    |
| Durgapur | 70    |

Roll-up to State:

| State       | Sales |
|-------------|-------|
| West Bengal | 220   |

Here:

City → State

We are aggregating data.

## Memory Trick

Roll-Up = Summary

Think:

Kolkata → West Bengal → India

Moving upward.

# 2. Drill-Down (Detailing)

## Definition

Drill-down is the opposite of roll-up. It moves from summarized data to more detailed data.

## Example

State Level:

| State       | Sales |
|-------------|-------|
| West Bengal | 220   |

Drill-down:

| City     | Sales |
|----------|-------|
| Kolkata  | 100   |
| Howrah   | 50    |
| Durgapur | 70    |

Here:

State → City

## Memory Trick

Drill-Down = More Details

Think:

India → West Bengal → Kolkata

Moving downward.

## 3. Slice

### Definition

Slice selects a single value from one dimension and creates a smaller sub-cube.

### Example

Original Cube:

Product × Region × Time

Suppose we choose:

Time = 2025

Now only 2025 data remains.

| Product | Region  | Sales |
|---------|---------|-------|
| Mobile  | Kolkata | 100   |
| Laptop  | Delhi   | 150   |

One dimension is fixed.

## Memory Trick

`Slice = One Condition`

Like slicing a cake at one position.

## 4. Dice

### Definition

Dice selects multiple values from multiple dimensions to create a smaller cube.

### Example

Select:

`Product = Mobile, Laptop`

`Region = Kolkata, Delhi`

`Year = 2025`

Now we get only records satisfying all conditions.

| Product | Region  | Year | Sales |
|---------|---------|------|-------|
| Mobile  | Kolkata | 2025 | 100   |
| Laptop  | Delhi   | 2025 | 150   |

Multiple dimensions are filtered.

## Memory Trick

`Dice = Multiple Conditions`

# GSP Algorithm (Generalized Sequential Pattern Algorithm)

GSP is a **Sequential Pattern Mining** algorithm used to discover frequent sequences in a database.

It was proposed by **Srikant and Agrawal** and is based on the **Apriori principle**.

## What is Sequential Pattern Mining?

Suppose customer purchase sequences are:

Customer 1: Bread → Milk → Butter

Customer 2: Bread → Butter

Customer 3: Bread → Milk → Butter

GSP finds frequent sequences such as:

Bread → Milk

Milk → Butter

Bread → Milk → Butter

## Apriori Principle Used in GSP

The main idea is:

If a sequence is not frequent, then none of its super-sequences can be frequent.

Example:

A → B

is infrequent.

Then:

A → B → C

A → B → D

cannot be frequent.

This helps reduce the search space.

# Steps of GSP Algorithm

## Step 1: Find Frequent 1-Sequences

Scan the database and find all items whose support is above the minimum support.

Example:

A : 60%

B : 70%

C : 20%

Minimum support = 30%

Frequent sequences:

{A}

{B}

C is discarded.

## Step 2: Generate Candidate 2-Sequences

Using frequent 1-sequences:

{A}

{B}

Generate:

A → B

B → A

## Step 3: Count Support

Scan database again.

Suppose:

A → B = 40%

B → A = 15%

Minimum support = 30%

Keep:

A → B

Remove:

$B \rightarrow A$

### Step 4: Generate Candidate 3-Sequences

Use frequent 2-sequences to generate longer sequences.

Example:

$A \rightarrow B$

$B \rightarrow C$

Generate:

$A \rightarrow B \rightarrow C$

### Step 5: Repeat Until No More Frequent Sequences

Continue:

Generate Candidates

↓

Count Support

↓

Prune Infrequent Sequences

↓

Repeat

Until no new frequent sequences are found.

## ROCK vs CACTUS

| Feature            | ROCK                                          | CACTUS                                      |
|--------------------|-----------------------------------------------|---------------------------------------------|
| Full Form          | Robust Clustering using Links                 | Clustering Categorical Data Using Summaries |
| Data Type          | Categorical Data                              | Categorical Data                            |
| Clustering Type    | Hierarchical Clustering                       | Partitioning/Subspace Clustering            |
| Main Idea          | Uses links (common neighbors) between objects | Uses attribute-value summaries              |
| Similarity Measure | Number of common neighbors (links)            | Attribute matching patterns                 |
| Suitable For       | Data with strong neighborhood relationships   | High-dimensional categorical data           |
| Complexity         | Higher                                        | Lower                                       |
| Output             | Hierarchical clusters                         | Flat clusters                               |

ROCK (Robust Clustering using Links) is a hierarchical clustering algorithm designed for categorical data. It groups objects based on the number of common neighbors or links

between them. CACTUS (Clustering Categorical Data Using Summaries) is a clustering algorithm that discovers clusters by analyzing frequent attribute-value combinations and summary information from categorical datasets. ROCK focuses on neighborhood relationships, whereas CACTUS focuses on attribute summaries. ROCK generally produces more robust clusters, while CACTUS is more efficient for large categorical datasets.

## GSP vs SPADE

| Feature              | GSP                                | SPADE                                                  |
|----------------------|------------------------------------|--------------------------------------------------------|
| Full Form            | Generalized Sequential Pattern     | Sequential Pattern Discovery using Equivalence Classes |
| Approach             | Apriori-based                      | Vertical ID-list based                                 |
| Database Scans       | Multiple scans                     | Very few scans                                         |
| Candidate Generation | Generates many candidates          | Generates fewer candidates                             |
| Storage Format       | Horizontal format                  | Vertical format                                        |
| Speed                | Slower                             | Faster                                                 |
| Memory Usage         | Lower                              | Higher                                                 |
| Efficiency           | Less efficient for large databases | More efficient for large databases                     |

## WUM (Web Usage Mining)

**Web Usage Mining** is a type of web mining that focuses on discovering user behavior patterns from web usage data.

### Definition

Web Usage Mining is the process of extracting useful information and patterns from web logs, clickstreams, browser history, cookies, and user interaction data.

## What Data is Used?

WUM analyzes:

- Web server logs
- Browser logs
- User sessions
- Clickstream data
- Cookies
- User profiles

Example log:

| User | Pages Visited          |
|------|------------------------|
| U1   | Home → Products → Cart |
| U2   | Home → Products        |
| U3   | Home → Products → Cart |

From this, WUM can discover:

Many users visit:  
Home → Products → Cart

## Process of Web Usage Mining

### 1. Data Collection

Collect web logs and user activity data.

↓

### 2. Data Preprocessing

- Remove irrelevant entries
- Session identification
- User identification

↓

### 3. Pattern Discovery

Apply:

- Association Rules
- Clustering
- Classification
- Sequential Pattern Mining

↓

### 4. Pattern Analysis



# C4.5 Algorithm

**C4.5** is a Decision Tree Classification algorithm developed by **Ross Quinlan** as an improvement over **ID3**.

## Definition

C4.5 is a supervised classification algorithm that constructs a decision tree using **Gain Ratio** as the splitting criterion.

## Why C4.5 was Developed?

Problem with ID3:

ID3 uses Information Gain

Information Gain tends to favor attributes having many distinct values.

Example:

Student\_ID

Every student has a unique ID.

ID3 may incorrectly choose it as the best split.

C4.5 solves this using **Gain Ratio**.

## Working of C4.5

### Step 1

Calculate Gain Ratio for every attribute.

↓

### Step 2

Choose attribute with highest Gain Ratio.

↓

### Step 3

Split the dataset.

↓

### Step 4

Repeat recursively for each child node.

↓

### Step 5

Stop when:

- All records belong to same class
- No attributes remain
- Dataset becomes empty

↓

### Step 6

Apply pruning if necessary.

## Backpropagation Algorithm

Backpropagation is a **supervised learning algorithm** used to train **Artificial Neural Networks (ANNs)**.

### Definition

Backpropagation is a learning algorithm that adjusts the weights of a neural network by propagating the error backward from the output layer to the input layer.

The main goal is:

Reduce Error

↓

Update Weights

↓

Improve Prediction

## Why is it Called Backpropagation?

Because the error travels:

Output Layer

↑

Hidden Layer  
↑  
Input Layer

That is, the error is propagated **backward** through the network.

## Neural Network Structure

Input Layer  
↓  
Hidden Layer  
↓  
Output Layer

Example:

Student Marks  
Attendance  
↓  
Hidden Neurons  
↓  
Pass / Fail

## Working of Backpropagation

### Step 1: Initialize Weights

Assign random weights.

Input → Hidden → Output

### Step 2: Forward Propagation

Input passes through the network.

Example:

Input = 5

Network predicts:

Output = 0.8

### Step 3: Calculate Error

Compare predicted output with actual output.

Actual Output = 1

Predicted Output = 0.8

$$\text{Error} = 1 - 0.8 = 0.2$$

### Step 4: Backward Propagation

Send the error backward through the network.

The network determines:

Which weights caused the error?

### Step 5: Update Weights

Weights are adjusted using:

$$\begin{aligned} \text{New Weight} \\ = \\ \text{Old Weight} \\ + \\ \text{Learning Rate} \times \text{Error} \times \text{Input} \end{aligned}$$

The objective is to minimize the error.

### Step 6: Repeat

```
Forward Pass
  ↓
Calculate Error
  ↓
Backward Pass
  ↓
Update Weights
  ↓
Repeat
```

until the error becomes very small.

# What is Web Mining?

## Definition

Web Mining is the process of discovering useful information, patterns, and knowledge from web data such as web pages, hyperlinks, and user browsing activities.

It applies **Data Mining techniques to web data**.

## Types of Web Mining

### 1. Web Content Mining

Extracts information from web page contents.

Examples:

- Text
- Images
- Videos
- Documents

Example:

News article analysis  
Product review analysis

### 2. Web Structure Mining

Analyzes the structure of hyperlinks between web pages.

Example:

Google PageRank

Studies:

Page A → Page B  
Page B → Page C

### 3. Web Usage Mining (WUM)

Analyzes user browsing behavior.

Example:

Home → Products → Cart → Payment

Used in:

- Recommendations
- Website optimization
- User behavior analysis

## Process of Web Mining

Web Data Collection  
↓  
Data Preprocessing  
↓  
Pattern Discovery  
↓  
Pattern Analysis  
↓  
Knowledge Extraction

## Applications of Web Mining

- Search Engines
- Recommendation Systems
- E-Commerce
- Customer Behavior Analysis
- Personalized Advertising
- Fraud Detection

## Web Mining vs Data Mining

| Feature     | Data Mining                                   | Web Mining                                |
|-------------|-----------------------------------------------|-------------------------------------------|
| Definition  | Extracts patterns from large datasets         | Extracts patterns from web data           |
| Data Source | Databases, Data Warehouses                    | Websites, Web Logs, Hyperlinks            |
| Data Type   | Structured data                               | Structured, Semi-structured, Unstructured |
| Scope       | General-purpose mining                        | Web-specific mining                       |
| Examples    | Sales analysis, Customer segmentation         | User click analysis, Page ranking         |
| Complexity  | Lower                                         | Higher                                    |
| Techniques  | Classification, Clustering, Association Rules | Same techniques + Web-specific analysis   |

# OLTP vs OLAP

| Feature         | OLTP                          | OLAP                              |
|-----------------|-------------------------------|-----------------------------------|
| Full Form       | Online Transaction Processing | Online Analytical Processing      |
| Purpose         | Daily business operations     | Data analysis and decision making |
| Data            | Current operational data      | Historical and summarized data    |
| Users           | Clerks, Cashiers, Customers   | Managers, Analysts, Executives    |
| Operations      | Insert, Update, Delete        | Query, Analysis, Reporting        |
| Queries         | Simple and short              | Complex and long                  |
| Response Time   | Milliseconds                  | Seconds/Minutes                   |
| Database Design | Highly normalized             | Denormalized (Star/Snowflake)     |
| Data Volume     | Detailed records              | Aggregated data                   |
| Example         | ATM, Railway Booking, Banking | Sales trend analysis, Forecasting |

## Web Structure Mining

### Definition

Web Structure Mining is the process of discovering useful information from the **link structure of websites** by analyzing the relationships between web pages through hyperlinks.

It focuses on **how pages are connected**, not on the content of the pages.

Consider web pages as nodes and hyperlinks as connections.

```
Page A -----> Page B
      |
      |
      v
Page C -----> Page D
```

Web Structure Mining analyzes these links to find important pages and relationships.

## What Does It Analyze?

### 1. Hyperlinks

Page A → Page B

A link from one page to another.

### 2. Website Structure

Home

- Products
- Services
- Contact

How pages are organized.

### 3. Relationships Between Pages

Finding:

Which pages are important?  
Which pages are related?

**Arbor Essbase** (later called **Hyperion Essbase**) is a popular **MOLAP (Multidimensional OLAP)** server used in Data Warehousing and Business Intelligence.

#### Definition

Arbor Essbase (Extended Spreadsheet Database) is a multidimensional database management system that stores data in OLAP cubes and allows fast analytical processing.

It was originally developed by **Arbor Software Corporation** and later acquired by Hyperion, which was eventually acquired by Oracle Corporation.

### Why is Essbase Used?

Organizations use Essbase for:

- Financial analysis
- Budgeting
- Forecasting
- Sales analysis
- Business reporting

### How Essbase Works

Instead of storing data in tables like ROLAP, Essbase stores data in **multidimensional cubes**.

Example cube:

```

          Time
          |
          |
Product ---+--- Region

```

Dimensions:

- Product



- Time
- Region

Measure:

- Sales

A manager can quickly ask:

Total laptop sales  
in Kolkata during 2025

and Essbase retrieves the answer from the cube very quickly.

## Features of Arbor Essbase

### 1. Multidimensional Storage

Stores data in cubes.

### 2. Fast Query Processing

Uses precomputed aggregations.

### 3. Supports OLAP Operations

- Roll-up
- Drill-down
- Slice
- Dice
- Pivot

### 4. Spreadsheet Integration

Works well with spreadsheet-style analysis.

### 5. Forecasting and Planning

Widely used in financial planning systems.

# Web-Enabled Data Warehousing

## Definition

Web-Enabled Data Warehousing is a data warehouse system that can be accessed and managed through the **World Wide Web (WWW)** using web browsers.

Instead of installing special software on every computer, users can access the data warehouse through a browser.

## Architecture

```
Data Sources
  ↓
ETL Process
  ↓
Data Warehouse
  ↓
Web Server
  ↓
Web Browser
  ↓
Users
```

### Example:

```
Manager opens Chrome
  ↓
Logs into company portal
  ↓
Views sales reports and dashboards
```

No special OLAP software is needed on the client machine.

## How It Works

### Step 1

Data is collected from:

- Databases
- ERP systems
- CRM systems
- Transaction systems

### Step 2

ETL processes clean and load the data into the warehouse.

### Step 3

A web server connects users to the warehouse.

### Step 4

Users access reports and OLAP tools through browsers.

## Mining Multimedia Data on the Web

### Definition

Mining Multimedia Data on the Web is the process of extracting useful information and patterns from multimedia content such as images, audio, video, animations, and graphics available on the web.

Unlike normal text mining, here the data is:

Images  
Videos  
Audio  
GIFs  
Graphics

### Why is it Needed?

The web contains huge amounts of multimedia data.

Example:

- YouTube videos
- Instagram images
- Spotify music
- CCTV footage

Mining helps discover useful patterns from them.

### Techniques Used

#### Image Mining

Extract information from images.

Example:

Face Recognition  
Object Detection  
Medical Image Analysis

## Video Mining

Extract patterns from videos.

Example:

Detecting suspicious activity  
Sports highlights generation  
Traffic monitoring

## Audio Mining

Analyze sound and speech.

Example:

Speech Recognition  
Voice Assistants  
Music Recommendation

## Applications

- ✓ Google Images
- ✓ YouTube Recommendations
- ✓ Face Recognition
- ✓ Medical Diagnosis
- ✓ Security Surveillance

## Example

Suppose YouTube analyzes:

Video Category  
Watch Duration  
User Preferences

and recommends similar videos.

This is multimedia mining.

# Automatic Classification of Web Documents

## Definition

Automatic Classification of Web Documents is the process of automatically assigning web pages or documents into predefined categories using machine learning and data mining techniques.

Instead of humans classifying pages manually, the computer does it automatically.

## Example

Suppose we have web documents:

Document 1:  
"Football World Cup"

Document 2:  
"Stock Market Growth"

Document 3:  
"Machine Learning"

The system classifies them as:

Sports

Finance

Technology

automatically.

## Working

### Step 1

Collect web documents.

↓

### Step 2

Extract keywords/features.

↓

### Step 3

Train a classifier.

Examples:

- Decision Tree
- Naive Bayes
- SVM
- Neural Networks

↓

### Step 4

Assign category labels.

## Advantages

- ✓ Fast
- ✓ Handles huge numbers of web pages
- ✓ Improves search engine organization
- ✓ Reduces manual effort

## Applications

- Search Engines
- News Classification
- Spam Detection
- E-commerce Product Categorization
- Digital Libraries

# Graph Mining

## Definition

Graph Mining is the process of discovering useful patterns, relationships, and knowledge from graph-structured data.

A graph consists of:

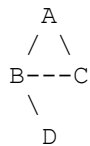
Vertices (Nodes)  
+  
Edges (Links)

Examples:

- Social networks
- Web links
- Transportation networks
- Biological networks
- Communication networks

## Graph Structure

Example:



Here:

A, B, C, D = Nodes

AB, AC, BC, CD = Edges

Graph mining analyzes these connections.

## Why Graph Mining?

Traditional data mining works well on tables:

### Student Department

|    |     |
|----|-----|
| S1 | CSE |
| S2 | ECE |

But many real-world datasets are connected like networks.

Example:

Facebook Friends  
Web Pages  
Road Networks

These are better represented as graphs.

## Tasks in Graph Mining

### 1. Frequent Subgraph Mining

Find graph patterns that occur frequently.

Example:

A-B-C

appears repeatedly in many graphs.

### 2. Graph Classification

Classify entire graphs.

Example:

Chemical Compound  
↓  
Toxic / Non-Toxic

### 3. Graph Clustering

Group similar nodes or graphs together.

Example:

Social Network  
↓  
Friend Communities

### 4. Link Prediction

Predict future connections.

Example:

Facebook:  
People You May Know